Lab 6: Image Classification using CNN

Objective

The objective of this lab is to design, train, and deploy a Convolutional Neural Network (CNN) for multi-class image classification using PyTorch.
Students will build a custom CNN architecture from scratch, organize the training code into reusable modules (model definition, training loop, and helper utilities), train the model on the **Imagenette** dataset, evaluate its performance using loss/accuracy curves, and finally deploy the trained model as an interactive web app using Streamlit.

Problem Statement

Imagenette is a smaller, easier-to-train subset of ImageNet containing 10 easily distinguishable classes (e.g., tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, parachute). Unlike the toy 2D datasets used in earlier labs, this is a real-world **image classification** problem, requiring a network that can extract spatial features from RGB images.
The goal is to build a **Custom CNN** that:

- Uses stacked convolutional layers to progressively extract low-level to high-level visual features.
- Uses `MaxPool2d` to downsample feature maps and reduce spatial dimensions.
- Flattens the final feature maps and passes them through fully-connected layers to produce class logits.
- Is trained using `CrossEntropyLoss` and the `Adam` optimizer, with the best-performing model (by test accuracy) saved to disk automatically.

Because a real project like this needs to be maintainable, the code is split into separate modules rather than a single notebook:

File	Purpose
model.py	Defines the CustomCNN architecture
trainNN.py	Contains reusable train_step, test_step, and train functions
helper_functions.py	Contains plotting utilities (e.g., plot_loss_curves)
training_notebook.ipynb	Loads the data, instantiates the model, and runs training
updated_GUI.py	A Streamlit app for deploying the trained model for live inference

Part -A: CustomCNN

Step 1: Import PyTorch and Check Device

```
import torch

print(f"Using torch {torch.__version__}")
if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
print(f"Using device = {device}")
```

Step 2: Set Dataset Directories

```
train_dir = 'data/train'
test_dir =  'data/val'
```

Step 3: Define Image Transforms

```
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

train_transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229,
     0.224, 0.225))
     ])
test_transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229,
     0.224, 0.225))
     ])
```

Step 4: Load the Imagenette Dataset

```
train_data = datasets.ImageFolder(root=train_dir,
transform=train_transform)
test_data = datasets.ImageFolder(root=test_dir,
transform=test_transform)
```

Step 5: Create DataLoaders

```
batch_size = 32
train_loader = DataLoader(train_data, batch_size=batch_size,
shuffle=True)
test_loader = DataLoader(test_data, batch_size=batch_size,
shuffle=False)
```

Step 6: Inspect the Dataset

```
print(f'Training data contains {len(train_data)} images')
print(f'Testing data contains {len(test_data)} images')

class_names = train_data.classes
print(class_names)

class_names_idx = train_data.class_to_idx
print(class_names_idx)
```

Step 7: Import and Instantiate the Model

```
from model import CustomCNN

num_classes = len(train_data.classes)
model = CustomCNN(num_classes).to(device)
```

Step 8: Print a Model Summary

```
from torchinfo import summary

model.to(device)
summary(model=model, input_size=(32,3,128,128)
col_names=["input_size", "output_size", "num_params", "trainable"],
col_width=20,row_settings=["var_names"])
```

Step 9: Define Loss Function and Optimizer

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
weight_decay=1e-4)
```

Step 10: Train the Model

```
torch.manual_seed(42)
torch.cuda.manual_seed(42)

from timeit import default_timer as timer
from trainNN import train
start_time = timer()

results = train(
    model=model,
    train_dataloader=train_loader,
    test_dataloader=test_loader,
    optimizer=optimizer,
    loss_fn=criterion,
    epochs=30,
    device=device
)

end_time = timer()
print(f"[INFO] Total training time: {end_time - start_time} seconds")
```

Step 11: Plot Loss and Accuracy Curves

```
from helper_functions import plot_loss_curves
plot_loss_curves(results)
```

Part -B: Transfer Learning with a Pretrained Network

Step 1: Load a Pretrained EfficientNet via timm

```
import torch
import torchvision.models as models
from torchvision.models import EfficientNet_B0_Weights
import timm

# Set device (GPU, MPS, or CPU)
device = "cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu"
print(f"Using device: {device}")

# Model
model = timm.create_model('efficientnet_b0', pretrained=True,
num_classes=10)
model.to(device)

# Move the model to the chosen device
model = model.to(device)
```

Step 2: Set Dataset Directories

```
train_dir = "data/train"
test_dir = "data/val"
```

Step 3: Define Transforms and DataLoaders

```
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader
from torch import nn

# Define transformations
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
     0.225])
     ])
```

```
# Load dataset
train_dataset = datasets.ImageFolder(root=train_dir,
transform=transform)
test_dataset = datasets.ImageFolder(root=test_dir,
transform=transform)

# Create DataLoaders
train_dataloader = DataLoader(train_dataset, batch_size=32,
shuffle=True)
test_dataloader = DataLoader(test_dataset, batch_size=32,
shuffle=False)
```

Step 4: Print a Model Summary

```
from torchinfo import summary

# Ensure model is on the correct device
model.to(device)

# Print model summary
summary(model=model,input_size=(32, 3, 224, 224),
col_names=["input_size", "output_size", "num_params", "trainable"],
col_width=20,
row_settings=["var_names"])
```

Step 5: Define Loss Function and Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
weight_decay=1e-4)
```

Step 6: Fine-tune the Model Using the Existing train() Function

```
from trainNN import train

# Train the model for 5 epochs
results = train(
    model=model,
    train_dataloader=train_dataloader,
    test_dataloader=test_dataloader,
```

```
    optimizer=optimizer,
    loss_fn=loss_fn,
    epochs=5,
    device=device
)
```

Step 7: Plot the Loss and Accuracy Curves

```
from helper_functions import plot_loss_curves
plot_loss_curves(results)
```

Tasks

- Train the CustomCNN on the Imagenette dataset for 30 epochs and record the final
- Plot the loss and accuracy curves using **plot_loss_curves** and comment on whether the model is overfitting, underfitting, or well-fit.
- Replace Adam with SGD (with momentum) and compare training stability and convergence speed.
- Increase or decrease **batch_size** (e.g., 16, 32, 64) and observe the effect on training speed and accuracy.
- Add data augmentation (e.g., RandomHorizontalFlip, RandomRotation) to train_transform and evaluate whether it improves generalization.
- Load **best_model.pth** in a fresh script/notebook and verify that predictions on a few test images match the results obtained during training.
- Use the trained model with updated_GUI.py to classify at least five of your own images (not from the Imagenette dataset) and report the predicted class and confidence for each.
- Identify at least one image where the model makes an incorrect prediction, display the class probability bar chart, and discuss possible reasons for the misclassification.